# From Single-Stream to Disentangled Factors: A Multi-View Framework for QoS Prediction

Haokai Ren, Zhichun Zhao, Yicong Ge, Zhixiang Feng, and Ding Ding

**Abstract**—Accurate quality of service (QoS) prediction supports personalized Web service selection and service management. Severe observation sparsity and heterogeneous influences make prediction challenging when all evidence is encoded in a single representation. We propose a disentangled multi-view framework (DMVF) for sparse time-aware QoS prediction. The time-independent view models persistent interactions using collaborative factors, entity identities, and interaction statistics. The time-dependent view combines absolute-time embeddings with compact summaries of nearby observations. The semantic view uses frozen language-model embeddings of geographic and network metadata to retrieve training observations from related users and services at the target time slot. Frozen text similarity selects eligible neighbors, and a trainable adapter refines their aggregation weights. Statistics of the retrieved QoS residuals are fused with multi-frequency geographic encodings to construct semantic tokens. Hierarchical attention integrates the three views, and a residual predictor corrects a median anchor. Experiments on the WS-DREAM dataset show competitive performance and improvements over several evaluated baselines. At 10% observation density, DMVF achieves the lowest RMSE among the reported methods, while STF retains lower MAE across all four evaluated densities.

**Index Terms**—Disentangled representation learning, QoS prediction, large language models, semantic context

## 1 Introduction

### 1.1 QoS Prediction in Service-Oriented Ecosystems

Service-oriented ecosystems allow users and applications to select and combine Web services according to their functional requirements. When multiple services provide similar functionality, their nonfunctional characteristics become essential selection criteria. Quality of Service (QoS), including response time and throughput, provides a quantitative basis for distinguishing these alternatives and supporting personalized service recommendation. Predicting the quality that a particular user would experience can therefore inform service selection without requiring that user to invoke every candidate service [1].

However, QoS depends on both the interacting entities and the conditions under which an invocation occurs. Geographical location and network connectivity contribute to differences across user–service pairs, while changing workloads and network conditions introduce temporal variation. Collecting measurements for every combination of user, service, and invocation time is costly, leaving incomplete observations from which these influences must be inferred. Real-world service measurements provide an empirical foundation for investigating this problem [2]. In this work, we consider time-aware QoS prediction as the estimation of missing entries in a user–service–time tensor, using available invocation records and contextual metadata.

### 1.2 Challenges and Motivation

Previous studies have explored several complementary approaches to this task. STCA [3] models interactions between spatial attributes and invocation time, while SCATSF [4] combines temporal neighborhood selection with spatial context and sequence modeling. More recently, STF [5] explicitly separates steady and transient QoS components across multiple prediction perspectives. Language models offer another source of information: llmQoS [6] encodes descriptions of users and services and combines the resulting features with historical interactions. These developments motivate a framework that jointly organizes persistent interaction patterns, temporal variation, and semantic context while accounting for their different characteristics.

First, persistent patterns and temporal changes require different treatments under sparse observations. Aggregating measurements across invocation times provides evidence about the typical quality of a user–service pair, but this aggregation can obscure local fluctuations. Conversely, nearby observations can reveal temporary changes, yet may be insufficient for estimating the underlying quality level when only a few measurements are available. A useful representation should preserve both forms of evidence and allow their contributions to depend on the information available for each prediction.

Second, interaction records and contextual metadata describe different aspects of service quality. Collaborative representations summarize patterns learned from observed invocations. Attributes such as location and autonomous-system affiliation describe the environments in which those invocations occur. Although both persistent interaction features and contextual attributes can remain unchanged across time, they have different sources and modeling roles. Pretrained language models provide a way to encode textual descriptions of these attributes, but translating such representations into useful QoS evidence still requires adaptation to the prediction task. Contextual similarity alone does not determine the response time of a particular invocation.

*This work is supported by Science and Technology Research and Development Plan of China Railway (P2025X001). The source code is available at https://github.com/AC-9217/DMVF. (Corresponding author: Ding Ding.)*
*The authors are with Beijing Jiaotong University, Beijing 100044, China (e-mail: 25120359@bjtu.edu.cn; 26110137@bjtu.edu.cn; 25120394@bjtu.edu.cn; 26125176@bjtu.edu.cn; dding@bjtu.edu.cn).*

Third, heterogeneous evidence differs in both granularity and availability. Entity representations characterize individual users or services, pairwise statistics summarize their interactions, and local observations describe a limited temporal neighborhood. Some predictions have informative neighboring measurements, whereas others depend more heavily on persistent patterns or metadata. These differences motivate separate processing within each evidence source, followed by interactions across sources. The fusion mechanism must also distinguish unavailable information from observed values so that missing context does not become a misleading prediction signal.

### 1.3 Research Contributions

To address these challenges, we propose a **Disentangled Multi-View Framework (DMVF)** for time-aware QoS prediction. DMVF organizes the available evidence into three complementary views: time-independent, time-dependent, and semantic. Here, disentanglement refers to separating modeling responsibilities through dedicated representations while retaining interactions between views. The framework first integrates features within each view and then combines their representations to estimate a correction to an empirical QoS baseline.

1) Structured modeling through three complementary views. The time-independent view combines collaborative factors, entity identities, and pairwise statistics to represent persistent interaction patterns. The time-dependent view combines absolute-time representations with local observation summaries to characterize shared temporal tendencies and nearby changes. The semantic view combines geographic encodings with statistics of training observations retrieved using metadata-based semantic similarity. This division gives each source of evidence an explicit role within the framework.
2) Language model-guided observation retrieval with geographic encoding. A frozen pretrained language model encodes geographic and network descriptions of users and services offline. Frozen text similarity selects eligible user and service neighbors with training observations at the target time slot, and a trainable adapter refines their aggregation weights. Retrieved observations are expressed as residuals relative to pairwise reference medians and summarized through weighted residual and support statistics. Separate projections combine these retrieval statistics with multi-frequency geographic encodings to form user and service semantic tokens. The retrieval library contains only training observations and excludes the target user-service pair. Coordinate-availability indicators handle missing geographic metadata, while paths without eligible supporting observations are masked during fusion.
3) Statistical compression of sparse local temporal context. We develop a compact temporal representation for sparse QoS observations by summarizing the available measurements within a local window, excluding the target time slot. The summary captures observation coverage, central tendency, dispersion, nearest-observation value and temporal distance, and a distance-weighted mean. These complementary statistics are projected into a single local token and integrated with an absolute-time embedding to jointly represent local QoS variation and temporal position.

The empirical focus is response-time prediction on WS-DREAM, using mean absolute error and root mean square error to assess the recovery of unobserved QoS values. The following sections review related work, formalize the prediction task, present DMVF, describe the experimental evaluation, and discuss its implications and limitations.

## 2 Related Work

### 2.1 Collaborative Filtering and Factorization for QoS Prediction

Collaborative filtering estimates unobserved QoS values by exploiting shared patterns in historical service invocations. Zheng et al. combine user-based and service-based predictions to support personalized Web service recommendation [1]. Their subsequent neighborhood-integrated matrix factorization approach combines neighborhood information with latent representations of users and services, bringing local collaborative relationships and global interaction patterns into a unified prediction model [7]. These approaches establish historical interactions as a primary source of evidence for personalized QoS estimation.

Tensor factorization extends collaborative modeling to time-varying observations. WSPred organizes invocation records into a user–service–time tensor and learns latent features along all three dimensions to recover missing QoS values [8]. More recently, STF explicitly decomposes QoS into steady and transient components. It constructs predictors from user, service, user–service pair, and user–service–time triplet perspectives, and combines their outputs through learned weights [5]. These developments demonstrate that temporal structure and complementary prediction perspectives are already important considerations in QoS modeling.

DMVF builds on these ideas by organizing input evidence according to its modeling role. Collaborative factors, entity identities, and pairwise statistics form the time-independent view, while temporal observations and contextual metadata receive dedicated representations. The resulting views interact before producing the final prediction, allowing persistent collaborative patterns to be combined with local temporal evidence and semantic context.

### 2.2 Temporal and Spatial Context Modeling

Context-aware QoS prediction incorporates information about invocation time and the environments of users and services. STCA introduces hierarchical neural models that encode spatial attributes, construct second-order feature interactions, and combine spatial representations with temporal features. Its attention mechanism assigns different weights to spatial features during integration [3]. This work provides an explicit treatment of heterogeneous contextual interactions beyond user and service identities.

SCATSF further combines temporal neighborhood selection, spatial interaction learning, and recurrent sequence

modeling. It measures similarity through time-varying QoS fluctuations to select neighboring users and services. The contextual attributes of target entities and their neighbors are processed through a pairwise multilayer network, followed by a spatial context-aware gated recurrent unit for QoS prediction [4]. Its architecture therefore connects neighborhood information with both spatial relationships and temporal evolution.

These studies motivate joint consideration of spatial and temporal evidence. DMVF places particular emphasis on representing local temporal context when observations for a target user–service pair are sparse. Available neighboring measurements are summarized through observation coverage, distributional statistics, nearest-observation information, and distance-weighted aggregation. The resulting representation is integrated with an absolute-time embedding. DMVF addresses missing-entry recovery within an observed time range, where the local context may contain known training measurements before and after the target time.

### 2.3 Language Model-Assisted Semantic Modeling for Services

Language models provide a way to represent contextual attributes through natural-language descriptions. Liu et al. propose llmQoS, which converts user and service attributes into descriptive sentences and extracts representations using pretrained language models. These representations are projected and combined with entity embeddings for QoS prediction. Their implementation describes attributes such as country, autonomous-system affiliation, and service-provider information, while excluding numerical coordinates from the textual input [6]. This work establishes language-derived metadata representations as a complement to historical interaction information.

Dedicated embedding models, including Qwen3 Embedding, provide pretrained text representations that can be adapted to downstream tasks [9]. Continuous coordinates also have established representation methods: Tancik et al. show that Fourier feature mappings help neural networks represent high-frequency variation in low-dimensional inputs [10]. These findings motivate complementary treatment of descriptive metadata and numerical geographic information.

DMVF uses language-derived context to guide observation retrieval. Frozen language-model embeddings of geographic and network descriptions determine which eligible users and services provide supporting training observations at the target time slot. A trainable adapter refines the aggregation weights, and the retrieved observations are summarized as QoS-residual and support statistics. These summaries are fused with multi-frequency coordinate encodings to construct semantic tokens. The semantic view then interacts with the time-independent and time-dependent views, connecting semantically retrieved QoS evidence with persistent interaction patterns and local temporal variation.

## 3 Problem Formulation

Let $\mathcal{U}$, $\mathcal{S}$, and $\mathcal{T}$ denote the sets of users, services, and time slots, respectively. Each service invocation is represented by a triple $(u, s, t)$, and its observed response time is denoted by $q_{ust}$.

The available invocation records are divided into a training set $\mathcal{Q}^{\text{train}}$ and a test set $\mathcal{Q}^{\text{test}}$. Each set contains records of the form $(u, s, t, q_{ust})$. The two sets are mutually exclusive:

$$\mathcal{Q}^{\text{train}} \cap \mathcal{Q}^{\text{test}} = \varnothing.$$

Specifically, each user–service–time triple belongs to only one of the two sets. Response times in $\mathcal{Q}^{\text{train}}$ are available for model learning and feature construction, while those in $\mathcal{Q}^{\text{test}}$ are reserved for evaluation.

Each user $u$ and service $s$ also has contextual attributes, denoted by $\mathbf{x}_u$ and $\mathbf{x}_s$. These attributes describe their geographic and network environments, including coordinates, location information, IP prefixes, and autonomous-system affiliations.

Given the training records and contextual attributes, the objective is to learn a model that predicts the response time of each test invocation:

$$\widehat{q}_{ust} = f_\theta\big(u, s, t, \mathbf{x}_u, \mathbf{x}_s; \mathcal{Q}^{\text{train}}\big)\,,$$

where $\theta$ denotes the trainable parameters. Prediction is performed within the observed time range. Local temporal context is drawn from training records of the same user–service pair before and after the target time, excluding the target entry itself. Performance is measured by comparing predictions with the response times in $\mathcal{Q}^{\text{test}}$, using mean absolute error and root mean square error.

## 4 Methodology

### 4.1 Framework Overview

DMVF consists of three view encoders, a hierarchical fusion module, and a residual prediction head (Figure 1). For a target invocation $(u, s, t)$, the framework uses records from $\mathcal{Q}^{\text{train}}$ and contextual attributes $\mathbf{x}_u$ and $\mathbf{x}_s$ to construct complementary representations of persistent interaction patterns, temporal variation, and semantic context.

The **time-independent view** combines collaborative factors learned from temporally aggregated QoS observations with entity identities and pairwise statistics. These features describe the persistent characteristics of users, services, and their interactions. The **time-dependent view** combines an absolute-time embedding with a compact statistical summary of nearby observations from the same user–service pair. It captures both temporal position and local QoS variation. The **semantic view** combines geographic encoding with language model-guided observation retrieval. Frozen text embeddings select eligible user and service neighbors with training observations at the target time slot, while a trainable adapter refines their aggregation weights. Statistics of the retrieved QoS residuals and supporting evidence are fused with multi-frequency geographic encodings to produce user and service semantic tokens.

Each view produces feature tokens with a shared embedding dimension. Fusion proceeds in two stages. First, self-attention models interactions among tokens within each view. The resulting tokens are then concatenated and supplemented with embeddings that identify their source views

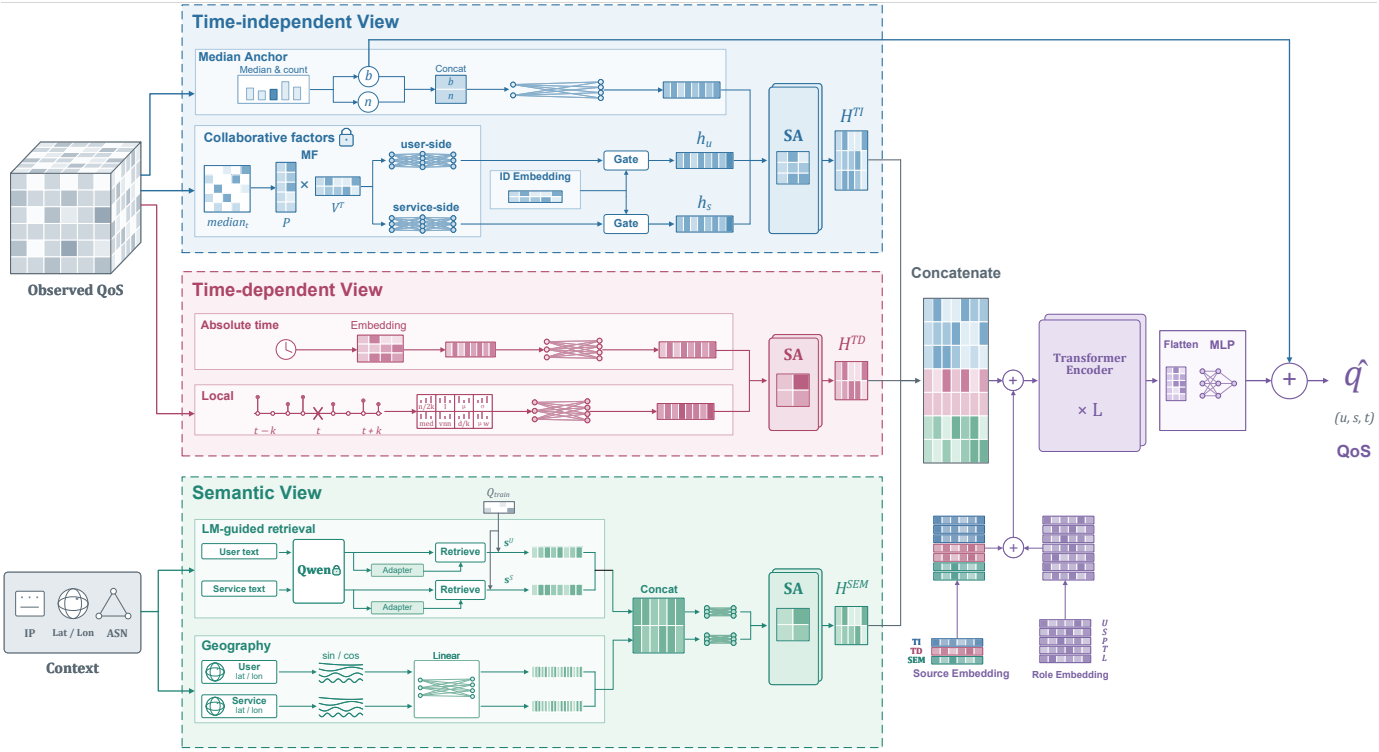



Fig. 1. Overview of DMVF. The time-independent and time-dependent views encode persistent interactions and temporal context. In the semantic view, Retrieve selects eligible same-slot training observations using frozen text similarity and weights their residuals using adapter-refined scores. Its outputs $\mathbf{s}^U_{ust}$ and $\mathbf{s}^S_{ust}$ summarize residuals and supporting evidence; geographic encodings and these summaries form the semantic tokens. Within-view self-attention (SA) precedes concatenation, source and role embeddings, and cross-view Transformer encoding. The MLP predicts a residual added to the median anchor. MF denotes matrix factorization.

and modeling roles. A second attention stage learns interactions across the three views, allowing the prediction to incorporate their complementary information.

Finally, a regression head maps the fused representations to a correction term $\Delta_{ust}$. The predicted response time is obtained by adding this correction to an empirical baseline:

$$\widehat{q}_{ust} = b_{ust} + \Delta_{ust},$$

where $b_{ust}$ denotes the median anchor and $\Delta_{ust}$ is the correction produced by the regression head. To prevent target leakage, the target measurement is excluded from the baseline calculation during training.

## 4.2 Time-Independent View

The time-independent view captures the long-term, stable characteristics of users and services. It integrates collaborative factors, entity ID embeddings, and statistics derived from user–service invocation records to construct three tokens. These tokens represent the long-term characteristics of the user, the service, and the user–service pair, respectively.

### *4.2.1 Temporal Aggregation and Collaborative Factorization*

For each user–service pair with available training observations, we use the median to aggregate information along the temporal dimension.

$$\bar{q}_{us} = \text{median}\left\{q_{us\tau} \mid (u, s, \tau, q_{us\tau}) \in \mathcal{Q}^{\text{train}}\right\}.$$

This aggregation produces a typical QoS value for each observed pair. We then apply matrix factorization to learn collaborative relationships among users and services. The estimated pairwise value is expressed as

$$\widetilde{q}_{us} = \mu + \alpha_u + \beta_s + \mathbf{p}_u^\top \mathbf{v}_s,$$

where $\mu$ is the mean of the observed pairwise medians, $\alpha_u$ and $\beta_s$ are user and service biases, and $\mathbf{p}_u, \mathbf{v}_s \in \mathbb{R}^r$ are latent factors of dimension $r$. The factors and biases are learned by minimizing the mean squared error over observed pairs with $L_2$ regularization.

After pretraining, the latent factors remain fixed during DMVF training. Separate trainable multilayer perceptrons project them into the shared token space, giving $\mathbf{m}_u = \phi_U(\mathbf{p}_u)$ and $\mathbf{m}_s = \phi_S(\mathbf{v}_s)$, where $\mathbf{m}_u, \mathbf{m}_s \in \mathbb{R}^d$. Each projection consists of two linear layers with a ReLU activation between them. $\mathbf{m}_u$ and $\mathbf{m}_s$ represent the collaborative features of users and services learned from the aggregated interaction records, respectively.

### *4.2.2 Entity Identity and Interaction Statistics*

We construct two types of features: entity identity features and interaction statistics. Entity identity features capture the individual characteristics of users and services, while interaction statistics summarize the typical QoS level and the amount of available observation data for each user–service pair.

For entity identity features, we learn separate ID embedding tables for users and services. The embeddings retrieved

for user $u$ and service $s$ are linearly projected into the shared $d$-dimensional feature space, yielding $\mathbf{i}_u, \mathbf{i}_s \in \mathbb{R}^d$. These embeddings and their projection layers are jointly optimized during model training.

For interaction statistics, we encode the median anchor $b_{ust}$ and the observation count $n_{ust}$ into a user–service interaction token:

$$\mathbf{h}_{us,t} = \mathbf{W}_p \begin{bmatrix} b_{ust} \\ \log(1 + n_{ust}) \end{bmatrix} + \mathbf{c}_p,$$

where $\mathbf{W}_p \in \mathbb{R}^{d\times 2}$ and $\mathbf{c}_p \in \mathbb{R}^d$ are learnable projection parameters. The median anchor provides a reference QoS level for the interaction, and $n_{ust}$ denotes the number of training observations used to construct the pair's statistics. The logarithmic transformation moderates the scale of the count feature.

To prevent target leakage, the target record is excluded when computing both statistics during training. During inference, the statistics are computed from all available training observations of the corresponding user–service pair.

#### 4.2.3 Within-View Feature Fusion

We first integrate the projected ID features with the collaborative representations through a gated residual connection. For the user branch,

$$\mathbf{g}_u = \sigma\left(\mathbf{W}_g^U[\mathbf{m}_u; \mathbf{i}_u] + \mathbf{c}_g^U\right),$$
$$\mathbf{h}_u = \mathrm{LayerNorm}_U\left(\mathbf{m}_u + \mathbf{g}_u \odot \mathbf{i}_u\right),$$

where $\mathbf{W}_g^U \in \mathbb{R}^{d\times 2d}$ and $\mathbf{c}_g^U \in \mathbb{R}^d$ are learnable parameters, $\sigma$ denotes the sigmoid activation, $[\,;\,]$ denotes concatenation, and $\odot$ denotes element-wise multiplication. The gate $\mathbf{g}_u$ modulates the contribution of the ID features in each dimension, while layer normalization produces the user token $\mathbf{h}_u$. The service token $\mathbf{h}_s$ is computed analogously using separate gating and normalization parameters.

The user, service, and pair tokens are then stacked into a token matrix and processed by the within-view self-attention module. This module models interactions among the three representations and outputs $\mathbf{H}_{ust}^{\mathrm{TI}} \in \mathbb{R}^{3\times d}$ for subsequent cross-view fusion.

### 4.3 Time-Dependent View

The time-dependent view captures QoS variations associated with invocation time. It constructs two complementary features: an absolute-time representation that describes temporal patterns shared across user–service pairs, and a local temporal representation that summarizes nearby observations of a specific pair. These features are encoded into two tokens and integrated through within-view attention.

#### 4.3.1 Absolute Time Representation

We learn an embedding $\mathbf{e}_t \in \mathbb{R}^{d_t}$ for each time slot $t$, where $d_t$ is the time embedding dimension. A linear projection maps this embedding into the shared token space:

$$\mathbf{h}_t^{\mathrm{abs}} = \mathbf{W}_T\mathbf{e}_t + \mathbf{c}_T,$$

where $\mathbf{W}_T \in \mathbb{R}^{d\times d_t}$ and $\mathbf{c}_T \in \mathbb{R}^d$ are learnable parameters. The time embeddings and projection parameters are jointly optimized during training. Invocations occurring at the same time slot share this representation, allowing the model to learn temporal effects common to different users and services.

#### 4.3.2 Local Temporal Context and Observation Availability

Local temporal context characterizes the QoS conditions surrounding the target invocation. We represent this context using statistical features of neighboring observations, together with information about their availability and temporal proximity.

We first standardize the observed response times using the global mean $\mu_{\mathrm{tr}}$ and standard deviation $\sigma_{\mathrm{tr}}$ computed from $\mathcal{Q}^{\mathrm{train}}$:

$$z_{us\tau} = \frac{q_{us\tau} - \mu_{\mathrm{tr}}}{\max(\sigma_{\mathrm{tr}}, \epsilon)},$$

where $\epsilon > 0$ ensures numerical stability. The same normalization parameters are used for all user–service pairs.

For a target invocation $(u, s, t)$ and window radius $k$, the available neighboring time slots are defined as

$$\mathcal{N}_{ust} = \left\{ \tau \;\middle|\; \begin{array}{l} (u, s, \tau, q_{us\tau}) \in \mathcal{Q}^{\mathrm{train}}, \\ 0 < |\tau - t| \le k \end{array} \right\}.$$

This neighborhood contains available training observations before and after the target time. To prevent target leakage, the target time slot itself is excluded. Missing observations and positions outside the indexed time range do not contribute to the statistics.

Let $n = |\mathcal{N}_{ust}|$. We construct an eight-dimensional summary vector $\mathbf{s}_{ust}^{\mathrm{loc}}$ from the normalized neighboring observations. It contains the observation ratio $n/(2k)$, a binary availability indicator $\mathbb{1}[n > 0]$, the mean, standard deviation, median, nearest observed value, normalized distance to the nearest observation, and a distance-weighted mean. For the local median, the lower middle value is used when the number of observations is even.

A learnable linear projection converts the statistical summary into a local temporal token:

$$\mathbf{h}_{ust}^{\mathrm{loc}} = \mathbf{W}_L\mathbf{s}_{ust}^{\mathrm{loc}} + \mathbf{c}_L,$$

where $\mathbf{W}_L \in \mathbb{R}^{d\times 8}$ and $\mathbf{c}_L \in \mathbb{R}^d$. This token jointly represents the local QoS distribution, observation availability, and temporal proximity.

#### 4.3.3 Within-View Feature Fusion

The absolute-time token $\mathbf{h}_t^{\mathrm{abs}}$ and local temporal token $\mathbf{h}_{ust}^{\mathrm{loc}}$ are concatenated along the token dimension. Two learnable role embeddings, $\mathbf{r}_{\mathrm{abs}}, \mathbf{r}_{\mathrm{loc}} \in \mathbb{R}^d$, are added to their corresponding tokens to identify the absolute-time and local-context roles, respectively. These embeddings are jointly optimized during training. Within-view self-attention then models interactions between temporal position and local QoS context, yielding the time-dependent representation $\mathbf{H}_{ust}^{\mathrm{TD}} \in \mathbb{R}^{2\times d}$.

### 4.4 Semantic View

The semantic view combines geographic encodings with QoS evidence retrieved from semantically related entities at the target time slot. Frozen language-model embeddings select eligible user and service neighbors, and a trainable adapter refines their aggregation weights. Two semantic tokens fuse the resulting retrieval statistics with the geographic representations of the target user and service, respectively.

#### 4.4.1 Geographical Encoding and Metadata Verbalization

We represent entity context using two complementary forms: numerical encodings of geographic coordinates and textual descriptions of location and network attributes.

For an entity $e$, let $\varphi_e$ and $\lambda_e$ denote its latitude and longitude in degrees. We normalize the coordinates as $\widetilde{\varphi}_e = \varphi_e/90$ and $\widetilde{\lambda}_e = \lambda_e/180$. Using a frequency vector $\boldsymbol{\omega} = [2^0, \ldots, 2^{R-1}]^\top$, we construct the geographic encoding

$$\boldsymbol{\gamma}_e = \begin{bmatrix} \sin(\boldsymbol{\omega}\widetilde{\varphi}_e) \\ \cos(\boldsymbol{\omega}\widetilde{\varphi}_e) \\ \sin(\boldsymbol{\omega}\widetilde{\lambda}_e) \\ \cos(\boldsymbol{\omega}\widetilde{\lambda}_e) \end{bmatrix} \in \mathbb{R}^{4R},$$

where the trigonometric functions operate element-wise. These features represent coordinate variation at multiple spatial scales. We append a coordinate-availability indicator and apply a shared learnable linear projection to obtain $\mathbf{g}_e^{\mathrm{geo}} \in \mathbb{R}^{d_g}$. Entities with missing or invalid coordinates receive a zero geographic representation.

For textual metadata, we use a fixed template to combine the entity's IP prefix, country, state, city, and autonomous system number into a single description. Missing fields are represented by "Unknown." The same template is used for users and services, providing a consistent input format for language model encoding.

#### 4.4.2 Language Model-Guided Observation Retrieval

We use a frozen Qwen3-Embedding-4B encoder [9] to obtain a textual representation $\mathbf{l}_e$ for each entity. The final hidden state of the last valid token is extracted and $L_2$-normalized. These representations are computed offline and cached for model training and inference.

A shared trainable adapter maps the cached representations into $\mathbf{a}_e \in \mathbb{R}^d$. The adapter consists of two linear layers with a GELU activation and dropout between them, followed by layer normalization. For two entities $e$ and $j$ of the same type, we compute

$$c_{ej} = \cos(\mathbf{l}_e, \mathbf{l}_j),$$
$$\eta_{ej} = \frac{c_{ej} + \rho \cos(\mathbf{a}_e, \mathbf{a}_j)}{\tau_{\mathrm{sem}}},$$

where $c_{ej}$ is the frozen text similarity, $\tau_{\mathrm{sem}} > 0$ is a temperature parameter, and $\rho = 0.1$ controls the adapter contribution. The frozen similarity determines neighbor selection, while the adapted score determines aggregation weights.

For a target invocation $(u, s, t)$, we retrieve observations through two complementary paths. The user path retrieves training observations of other users invoking service $s$ at time $t$. The service path retrieves training observations of user $u$ invoking other services at the same time slot. Both the query entity and its candidate neighbors must have valid textual metadata.

We retain candidate observations only when their user–service pairs contain at least two training records. Among eligible entities with an available observation at $t$, we select the top $K$ neighbors according to frozen text similarity, retaining all ties at the selection boundary. The selected user and service neighbor sets are denoted by $\mathcal{N}_{ust}^U$ and $\mathcal{N}_{ust}^S$, respectively.

Each retrieved observation is expressed as a deviation from its own pair's reference QoS level. For a training observation $(a, b, t, q_{abt})$, this residual is

$$r_{abt} = q_{abt} - \mathrm{median}\left\{ q_{ab\tau} \mid (a, b, \tau, q_{ab\tau}) \in \mathcal{Q}^{\mathrm{train}},\ \tau \neq t \right\}.$$

The reference median excludes the retrieved observation itself. This representation describes its deviation from the pair's typical QoS level.

For the user path, the aggregation weights and weighted residual are

$$w_{u';ust}^U = \frac{\exp(\eta_{uu'})}{\sum_{v \in \mathcal{N}_{ust}^U} \exp(\eta_{uv})}, \qquad u' \in \mathcal{N}_{ust}^U,$$
$$\overline{r}_{ust}^U = \sum_{u' \in \mathcal{N}_{ust}^U} w_{u';ust}^U \, r_{u'st}.$$

The service path is computed analogously using service similarities and residuals $r_{us't}$.

For each path, we summarize the retrieved observations using the weighted residual mean and standard deviation, the log-transformed support count, the weighted frozen text similarity, and the weighted logarithmic count of the remaining training observations used to compute the reference medians. These statistics describe the retrieved QoS deviations and their supporting evidence, yielding summary vectors $\mathbf{s}_{ust}^U$ and $\mathbf{s}_{ust}^S$.

The retrieval library contains only records from $\mathcal{Q}^{\mathrm{train}}$. To prevent target leakage, both paths exclude the target user–service pair from their supporting observations.

#### 4.4.3 Within-View Feature Fusion

We combine the geographic representation of each target entity with the statistics from its corresponding retrieval path. Text representations affect these statistics through neighbor selection and aggregation weights. Separate learnable linear projections produce two semantic tokens:

$$\mathbf{h}_{ust}^{\mathrm{sem},U} = \mathbf{W}_{\mathrm{sem}}^U [\mathbf{g}_u^{\mathrm{geo}}; \mathbf{s}_{ust}^U] + \mathbf{c}_{\mathrm{sem}}^U,$$
$$\mathbf{h}_{ust}^{\mathrm{sem},S} = \mathbf{W}_{\mathrm{sem}}^S [\mathbf{g}_s^{\mathrm{geo}}; \mathbf{s}_{ust}^S] + \mathbf{c}_{\mathrm{sem}}^S,$$

where both tokens belong to $\mathbb{R}^d$. The user token integrates the target user's geographic context with observations from related users, while the service token integrates the target service's geographic context with observations from related services.

The two tokens are concatenated along the token dimension and supplemented with learnable role embeddings identifying the user and service retrieval paths. Within-view

self-attention models interactions between them, yielding the semantic representation $\mathbf{H}_{ust}^{\mathrm{SEM}} \in \mathbb{R}^{2\times d}$. A path without eligible supporting observations is masked during fusion, and its output token is set to zero.

### 4.5 Hierarchical View Fusion

Hierarchical view fusion integrates the three views through two stages of self-attention. Within-view fusion models interactions among features constructed by the same encoder. Cross-view fusion then combines their outputs to learn dependencies among long-term interaction patterns, temporal variation, and semantic context.

#### 4.5.1 Attention Mechanism for Within-View Fusion

Each view uses an independent multi-head self-attention block. Let $\mathbf{X}^v \in \mathbb{R}^{n_v\times d}$ denote the token matrix of view $v$ after adding its learnable role embeddings, where $n_{\mathrm{TI}} = 3$, $n_{\mathrm{TD}} = 2$, and $n_{\mathrm{SEM}} = 2$. The invocation indices are omitted here for clarity.

For attention head $h$, learned linear projections produce query, key, and value matrices $\mathbf{Q}_h$, $\mathbf{K}_h$, and $\mathbf{V}_h$. The attention output is

$$\mathbf{A}_h = \mathrm{softmax}\left(\frac{\mathbf{Q}_h\mathbf{K}_h^\top}{\sqrt{d_h}} + \mathbf{M}\right)\mathbf{V}_h,$$

where $d_h = d/H$, $H$ is the number of attention heads, and $\mathbf{M}$ is an additive availability mask. Its entries are zero for available key positions and $-\infty$ for unavailable positions. The head outputs are concatenated and linearly projected into the shared token space to obtain $\mathrm{MHA}(\mathbf{X}^v)$.

The attention output is processed through a residual connection and layer normalization, followed by a token-wise feed-forward network:

$$\widetilde{\mathbf{X}}^v = \mathrm{LayerNorm}_1\left(\mathbf{X}^v + \mathrm{Dropout}\big(\mathrm{MHA}(\mathbf{X}^v)\big)\right),$$
$$\mathbf{H}^v = \mathrm{LayerNorm}_2\left(\widetilde{\mathbf{X}}^v + \mathrm{Dropout}\big(\mathrm{FFN}(\widetilde{\mathbf{X}}^v)\big)\right).$$

The feed-forward network consists of two linear layers with a GELU activation and dropout between them. Dropout is also applied to the attention weights during training. Each view maintains separate attention, feed-forward, and normalization parameters.

Unavailable token positions are zeroed before the block and after its output. If a view contains no available tokens, its output is a zero matrix. This procedure preserves the token organization of each view while restricting information exchange to available features.

#### 4.5.2 Cross-View Interaction with Source and Role Embeddings

The outputs of the three views are concatenated along the token dimension:

$$\mathbf{Z}_{ust} = \mathrm{Concat}_{\mathrm{token}}\left(\mathbf{H}_{ust}^{\mathrm{TI}}, \mathbf{H}_{ust}^{\mathrm{TD}}, \mathbf{H}_{ust}^{\mathrm{SEM}}\right) \in \mathbb{R}^{7\times d}.$$

We introduce source and role embeddings to distinguish the origin and modeling function of each token. For token $i$, the input to cross-view fusion is

$$\mathbf{z}_i^{(0)} = \mathbf{z}_i + \mathbf{e}_{v(i)}^{\mathrm{src}} + \mathbf{e}_{r(i)}^{\mathrm{role}},$$

where $v(i)$ identifies the time-independent, time-dependent, or semantic view, and $r(i)$ identifies the user, service, interaction, absolute-time, or local-context role. The user and service tokens from the time-independent and semantic views share their respective role embeddings, while their source embeddings distinguish the views.

The resulting sequence is processed by $L$ stacked self-attention blocks with the same structure as the within-view block. Each block has independent parameters and allows all available tokens to interact. The availability masks are carried forward from the view encoders, and unavailable positions are zeroed after each block.

The final representation, $\mathbf{Z}_{ust}^{(L)} \in \mathbb{R}^{7\times d}$, retains the seven token positions and incorporates dependencies across the three views. It is subsequently used to predict the invocation-specific QoS correction.

### 4.6 Residual Prediction and Model Learning

DMVF predicts response time by learning a correction to a median anchor. The trainable view encoders, fusion modules, and regression head are jointly optimized using observed training records.

#### 4.6.1 Median Baseline and Residual Estimation

The median anchor $b_{ust}$ provides a stable reference QoS level. The main model learns to adjust this reference using the fused representations of the target invocation.

We flatten the cross-view representation and apply a two-layer regression network:

$$\Delta_{ust} = \mathbf{w}_o^\top \mathrm{Dropout}\left(\mathrm{ReLU}\left(\mathbf{W}_R\,\mathrm{vec}\big(\mathbf{Z}_{ust}^{(L)}\big) + \mathbf{c}_R\right)\right) + c_o,$$

where $\mathbf{W}_R \in \mathbb{R}^{d_p\times 7d}$, $\mathbf{c}_R, \mathbf{w}_o \in \mathbb{R}^{d_p}$, and $c_o \in \mathbb{R}$ are learnable parameters. The scalar correction can increase or decrease the reference value. The final prediction is

$$\widehat{q}_{ust} = b_{ust} + \Delta_{ust}.$$

#### 4.6.2 Training Objective and Observation Masking

We optimize the main model using the Huber loss. Let $e_{ust} = \widehat{q}_{ust} - q_{ust}$ denote the prediction error. For a threshold $\delta > 0$, the loss for an individual observation is

$$\ell_\delta(e) = \begin{cases} \dfrac{1}{2}e^2, & |e| \le \delta, \\ \delta\left(|e| - \dfrac{1}{2}\delta\right), & |e| > \delta. \end{cases}$$

This objective provides a quadratic penalty for small errors and a linear penalty for large errors. The regularized training objective is

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{Q}^{\mathrm{train}}|}\sum_{(u,s,t,q_{ust})\in\mathcal{Q}^{\mathrm{train}}} \ell_\delta(\widehat{q}_{ust} - q_{ust}) + \frac{1}{2}\sum_{g\in\mathcal{G}}\lambda_g\|\theta_g\|_2^2.$$

Here, $\mathcal{G}$ contains three parameter groups: user and service ID embeddings, absolute-time embeddings, and the linear weights of the semantic adapter and retrieval-statistic

projections. Each group has its own regularization coefficient $\lambda_g$. The remaining trainable parameters receive no explicit weight decay.

We use Adam to jointly optimize the embeddings, feature projections, gating layers, semantic adapter, attention modules, and regression head. The pretrained collaborative factors and cached language model representations remain fixed. Gradients reach the semantic adapter through the retrieval aggregation weights.

To prevent target leakage, the target record is excluded when computing its training anchor and interaction count. The local temporal summary excludes the target time slot, while semantic retrieval excludes the target user–service pair. Each retrieved residual also uses a reference median computed without the retrieved observation itself. All QoS statistics and retrieval records are constructed from $\mathcal{Q}^{\text{train}}$.

#### 4.6.3 Training and Inference Complexity

Before main-model training, we compute the offline language model representations, pretrain the collaborative factors, and construct the training observation caches. These caches contain the median anchors, local temporal observations, and semantic retrieval residuals. Mini-batch training then constructs the three views, performs hierarchical fusion, and updates the trainable parameters using the prediction loss.

During inference, dropout is disabled and the median anchor uses the available training records. Local context and semantic retrieval continue to access the training observation caches. Language model encoding and collaborative factorization are performed during preprocessing and require no repeated execution for individual predictions.

For a token sequence of length $n$, a self-attention block has computational cost

$$C(n) = O\big(nd^2 + n^2 d + ndd_f\big)\,,$$

where $d_f$ is the feed-forward hidden dimension. The hierarchical fusion cost per invocation is therefore

$$O\left(\sum_v n_v d^2 + \sum_v n_v^2 d + \sum_v n_v d d_f + L\left(nd^2 + n^2 d + ndd_f\right)\right)$$

with $n_v \in \{3, 2, 2\}$ and $n = 7$. The regression head adds $O(ndd_p)$ computation. The number of tokens remains fixed as the number of available observations changes.

Semantic retrieval additionally requires scoring candidate entities. Let $U = |\mathcal{U}|$, $S = |\mathcal{S}|$, $B$ be the batch size, $d_\ell$ the cached text embedding dimension, and $d_a$ the adapter hidden dimension. Applying the adapter to all entities and computing query-to-candidate scores costs

$$O((U+S)(d_\ell d_a + d_a d) + B(U+S)d)$$

per batch, followed by neighbor selection and aggregation. With retrieval restricted to the target time slot, each candidate pair contributes at most one supporting observation.

The main observation and semantic caches require

$$O\big(UST + U^2 + S^2 + (U+S)d_\ell\big)$$

storage, where $T = |\mathcal{T}|$. These terms correspond to the dense observation caches, frozen within-type similarity matrices, and cached text representations, respectively.

## 5 Experimental Evaluation

### 5.1 Experimental Setup

#### 5.1.1 Dataset and Preprocessing

We evaluate DMVF on **Supplement_WSDREAM_dataset**, the spatially enriched WS-DREAM dataset used by Zhou et al. [3]. The original dataset contains response-time measurements involving 420 users and 1,000 services across 480 time slots, with a sampling interval of 30 minutes. In our experiments, we retain the first 140 users, all 1,000 services, and the first 96 time slots according to their original indices, yielding a tensor of size $140 \times 1{,}000 \times 96$.

Preprocessing validates record structure, entity and time indices, and response-time values. Damaged indices are repaired only when the ordering of surrounding records uniquely determines the intended indices; ambiguous records are discarded. Duplicate records are resolved using index consistency, while equally reliable records with conflicting measurements are removed. We further discard records with missing or nonnumeric fields, out-of-range indices, or nonpositive response times. The resulting subset contains **12,071,042 valid observations**, covering 139 users and 992 services within the retained index space.

User and service metadata are aligned with their original entity indices. These attributes include geographic coordinates, country, state, city, IP prefixes, and autonomous-system affiliations. Coordinates are processed through the geographical encoder, while location and network attributes are verbalized for language model encoding. Missing contextual information is handled using the availability mechanisms described in Section 4.

#### 5.1.2 Data Splits, Observation Densities, and Evaluation Metrics

We consider four observation densities: **5%, 10%, 15%, and 20%**. At each density $\rho$, every valid observation is independently assigned to the training set $\mathcal{Q}^{\text{train}}$ with probability $\rho$, and the remaining observations form the test set $\mathcal{Q}^{\text{test}}$. The two sets are disjoint. Here, observation density refers to the sampling proportion of valid records. This protocol evaluates missing QoS prediction within the retained time range.

All QoS-dependent features, including collaborative factors, median anchors, interaction statistics, local temporal summaries, and semantic retrieval records, are constructed exclusively from $\mathcal{Q}^{\text{train}}$. Response times used in local temporal summaries are standardized using the mean and standard deviation of training observations. Prediction targets and evaluation metrics retain their original scale in seconds.

We measure prediction accuracy using mean absolute error (MAE) and root mean square error (RMSE):

$$\text{MAE} = \frac{1}{|\mathcal{Q}^{\text{test}}|} \sum_{(u,s,t,q_{ust}) \in \mathcal{Q}^{\text{test}}} |\widehat{q}_{ust} - q_{ust}|\,,$$

TABLE 1
Baseline methods and their descriptions.

| Method | Description |
|---|---|
| **RNCF** [11] | A recurrent neural collaborative filtering method that uses gated recurrent units to model temporal dependencies. |
| **WSPred** [8] | A time-aware prediction framework that factorizes user–service–time observations to estimate missing QoS values. |
| **DeepTSQP** [12] | A deep temporal QoS prediction model that combines invocation and similarity features with gated recurrent units. |
| **TUIPCC** [13] | A time-aware collaborative filtering method that combines temporal prediction with user- and service-based neighborhood information. |
| **SCATSF** [4] | A spatial context-aware forecasting model that integrates temporal neighborhood selection, spatial interactions, and recurrent sequence modeling. |
| **PLMF** [14] | A personalized LSTM-based matrix factorization method for online QoS prediction. |
| **CoSTCo** [15] | A convolutional tensor completion model that learns nonlinear interactions among latent representations. |
| **GM** [16] | A graph-based model that represents user–service–time relationships and learns their dependencies through graph convolution. |
| **NTF** [17] | A neural tensor factorization model that combines recurrent temporal modeling with nonlinear latent interactions. |
| **STF** [5] | A factorization framework that separates steady and transient QoS components and combines predictions from complementary user, service, pair, and triplet perspectives. |

$$\mathrm{RMSE} = \sqrt{\frac{1}{|\mathcal{Q}^{\mathrm{test}}|} \sum_{(u,s,t,q_{ust}) \in \mathcal{Q}^{\mathrm{test}}} (\widehat{q}_{ust} - q_{ust})^2}.$$

MAE measures the average absolute prediction error, whereas RMSE assigns a larger penalty to large deviations. Lower values indicate better predictive performance for both metrics.

#### 5.1.3 Baselines and Implementation Details

We consider ten representative baseline methods covering collaborative filtering, tensor factorization, recurrent modeling, and graph-based prediction (Table 1). Nine baseline implementations are evaluated using the same preprocessed observations and data partitions as DMVF. SCATSF results are taken from the original paper [4] and included as a literature reference under its reported experimental protocol.

DMVF is implemented using Python 3.12.14 and PyTorch 2.11.0 with CUDA 12.8, and runs on an NVIDIA RTX A4000 GPU. The shared token dimension is $d = 64$. Each view contains one internal self-attention block, followed by two cross-view attention blocks. All attention blocks use four heads, a feed-forward hidden dimension of 128, and a dropout rate of 0.25. The residual regression head has a hidden dimension of 128.

For the time-independent view, the collaborative factor dimension is 16, and the user and service ID embedding dimension is 16. Matrix factorization uses a learning rate of $10^{-2}$ and an $L_2$ regularization coefficient of $10^{-2}$. Its training duration is 100 epochs at 5% density, 140 epochs at 10% density, and 200 epochs at 15% and 20% densities. The learned collaborative factors remain fixed during main-model training. For the time-dependent view, the absolute-time embedding dimension is 16, and the local window extends 16 time slots on each side of the target time.

For the semantic view, geographic coordinates are encoded using eight sinusoidal frequencies and projected into 16-dimensional representations. Qwen3-Embedding-4B [9] generates 2,560-dimensional text representations offline and remains frozen. The semantic adapter uses a hidden dimension of 128 and a dropout rate of 0.1. Each retrieval path selects the top 16 eligible neighbors by frozen text similarity, retaining ties at the selection boundary. Aggregation uses adapter-refined scores with a softmax temperature of 0.1. Retrieval is restricted to the target time slot.

The main model is optimized for 50 epochs using Adam with a learning rate of $10^{-3}$ and a batch size of 8,192. We use the Huber loss with threshold $\delta = 1.0$. Weight decay is set to $10^{-2}$ for user and service ID embeddings, and $10^{-4}$ for absolute-time embeddings and the linear weights of the semantic adapter and retrieval-statistic projections.

### 5.2 Overall Prediction Performance

#### 5.2.1 Prediction Accuracy

Table 2 summarizes response-time prediction performance across four observation densities. DMVF achieves the second-lowest MAE at every density among the available results and remains competitive in RMSE.

DMVF consistently outperforms NTF, GM, CoSTCo, and PLMF in both metrics across all four densities. Compared with NTF, the strongest of these baselines in MAE, DMVF reduces MAE by **1.72%–4.51%** and RMSE by **3.16%–4.50%**. Relative to CoSTCo, the reductions reach **10.52%–12.02%** and **8.51%–10.05%**, respectively.

DMVF benefits from increasing observation density. Its MAE decreases monotonically from 0.3328 at 5% density to 0.3074 at 20%, corresponding to a **7.64%** reduction. RMSE decreases by **4.24%** between these endpoints, with a small fluctuation at 15%. At the sparsest setting, DMVF achieves the second-lowest value in both metrics among the available results, demonstrating competitive prediction performance even with limited training observations.

STF provides the strongest comparison in MAE. Although STF achieves lower MAE at every density, DMVF's relative gap remains within **2.26%** and narrows to **1.58%** at 20%. The RMSE comparison is more favorable to DMVF: it obtains lower values than STF at 10%, 15%, and 20% densities. At 10%, DMVF achieves the lowest RMSE among the available results, reducing the error from STF's 1.1664 to 1.1591. Thus, the difference between the two methods depends on the evaluation metric, with DMVF retaining closely comparable absolute error while improving squared-error performance at three densities.

#### 5.2.2 Computational Efficiency

We further compare computational efficiency at **10% observation density** and a **batch size of 8,192**, using the same

TABLE 2
Response-time prediction performance at different observation densities.

| Method | 5% | | 10% | | 15% | | 20% | |
|---|---|---|---|---|---|---|---|---|
| | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE |
| RNCF | 0.7151 | 1.7431 | 0.7106 | 1.7327 | 0.7128 | 1.7341 | | |
| WSPred | 0.5565 | 1.5169 | 0.5561 | 1.5152 | 0.5561 | 1.5154 | 0.5561 | 1.5160 |
| DeepTSQP | 0.6221 | 1.6113 | 0.5372 | 1.4558 | 0.4956 | 1.3934 | | |
| TUIPCC | 0.4381 | 1.2253 | 0.4119 | 1.1678 | 0.4022 | **1.1486** | 0.3963 | **1.1378** |
| SCATSF† | 0.4099 | 1.3059 | 0.3963 | 1.2309 | 0.3823 | 1.2298 | 0.3778 | 1.2124 |
| PLMF | 0.3915 | 1.3363 | 0.3809 | 1.3248 | 0.3813 | 1.3111 | 0.3786 | 1.3202 |
| CoSTCo | 0.3782 | 1.3031 | 0.3571 | 1.2886 | 0.3501 | 1.2695 | 0.3462 | 1.2647 |
| GM | 0.3523 | 1.2399 | 0.3431 | 1.2199 | 0.3450 | 1.2123 | 0.3399 | 1.2045 |
| NTF | 0.3386 | 1.2311 | 0.3307 | 1.2101 | 0.3245 | 1.2004 | 0.3219 | 1.1956 |
| STF | **0.3254** | **1.1888** | **0.3129** | 1.1664 | **0.3070** | 1.1614 | **0.3026** | 1.1500 |
| **DMVF (ours)** | 0.3328 | 1.1922 | 0.3196 | **1.1591** | 0.3121 | 1.1607 | 0.3074 | 1.1417 |

† Literature-reported results under the original experimental protocol.

GPU model. Table 3 reports prediction accuracy alongside the recorded batch throughput, with higher-throughput methods appearing lower in the table. DMVF and STF are measured during training. TUIPCC is included as a prediction-throughput reference because its implementation performs neighborhood-based prediction without gradient-based parameter training.

TABLE 3
Prediction accuracy and batch throughput at 10% observation density and a batch size of 8,192.

| Method | MAE ↓ | RMSE ↓ | Throughput (it/s) ↑ | Measured stage |
|---|---|---|---|---|
| TUIPCC | 0.4119 | 1.1678 | 6.17 | Prediction |
| STF | **0.3129** | 1.1664 | 7.68 | Training |
| **DMVF (ours)** | 0.3196 | **1.1591** | 7.76 | Training |

DMVF processes **7.76 training batches per second**, compared with **7.68** for STF, yielding a modest observed throughput increase of **1.04%**. Under the same density setting, DMVF's MAE is 2.13% higher, while its RMSE is 0.63% lower. These measurements show that DMVF maintains competitive prediction accuracy while providing slightly higher training throughput under the evaluated configuration.

Compared with TUIPCC, DMVF reduces MAE by **22.41%** and RMSE by **0.75%** at 10% density. TUIPCC's recorded throughput of 6.17 iterations per second provides an additional execution reference, although its prediction iterations are computationally different from the training iterations measured for DMVF and STF.

### 5.3 Ablation Studies

We examine the contributions of the three views at 10% observation density by comparing DMVF with three variants (Table 4): **w/o TI**, which removes the time-independent view; **w/o TD**, which removes the time-dependent view; and **w/o SEM**, which removes the semantic view. All variants use the same data partition and optimization settings. The median anchor remains available in every configuration, including w/o TI, so the comparison evaluates the contribution of each feature branch within the residual prediction framework.

Removing the **time-independent view** causes the largest degradation in both metrics. MAE increases from 0.3196 to 0.3378, while RMSE rises from 1.1591 to 1.2317, corresponding to relative increases of **5.69%** and **6.26%**, respectively. This variant removes the collaborative representations, entity identity features, and interaction-statistic token while retaining the median anchor. The degradation therefore highlights the contribution of the time-independent representations beyond the empirical reference level supplied by the anchor.

Removing the **time-dependent view** increases MAE by **1.58%** and RMSE by **1.98%**. This ablation jointly removes the absolute-time representation and the local temporal summary. The resulting degradation supports incorporating temporal position and nearby observations alongside persistent interaction features. Although its effect is smaller than that of removing the time-independent view, the temporal branch improves both evaluation metrics.

The **semantic view** exhibits a metric-dependent contribution. Removing this branch, including geographical encoding and language-model-guided observation retrieval, slightly decreases MAE from 0.3196 to 0.3192, a relative reduction of **0.11%**. However, RMSE increases from 1.1591 to 1.1681, or **0.78%**. Semantic information therefore provides a modest RMSE benefit in this setting, accompanied by a small MAE trade-off. Its observed contribution is most evident in the metric that assigns greater weight to large prediction deviations. The complete model achieves the lowest RMSE across all four configurations.

### 5.4 Hyperparameter Sensitivity

We examine six hyperparameters at 10% observation density with a batch size of 8,192 (Figure 2). The default architecture uses a token dimension of 128, a collaborative rank of 32, and a feed-forward hidden dimension of 512.

**Local window radius.** Increasing $k$ from 4 to 16 decreases MAE from 0.3237 to 0.3212. Further expansion provides no additional MAE improvement, with values remaining close for $k \in \{16, 24, 32\}$. RMSE follows a different pattern and reaches its minimum at $k = 4$. These results favor a moderate window for absolute-error minimization while showing that broader temporal coverage does not uniformly improve both metrics.

TABLE 4
Ablation results at 10% observation density.

| Variant | Time-independent | Time-dependent | Semantic | MAE ↓ | RMSE ↓ |
|---|---|---|---|---|---|
| w/o TI | — | ✓ | ✓ | 0.3378 | 1.2317 |
| w/o TD | ✓ | — | ✓ | 0.3246 | 1.1820 |
| w/o SEM | ✓ | ✓ | — | **0.3192** | 1.1681 |
| **DMVF** | ✓ | ✓ | ✓ | 0.3196 | **1.1591** |

MAE and RMSE are reported in seconds. Bold values indicate the best result in each column.

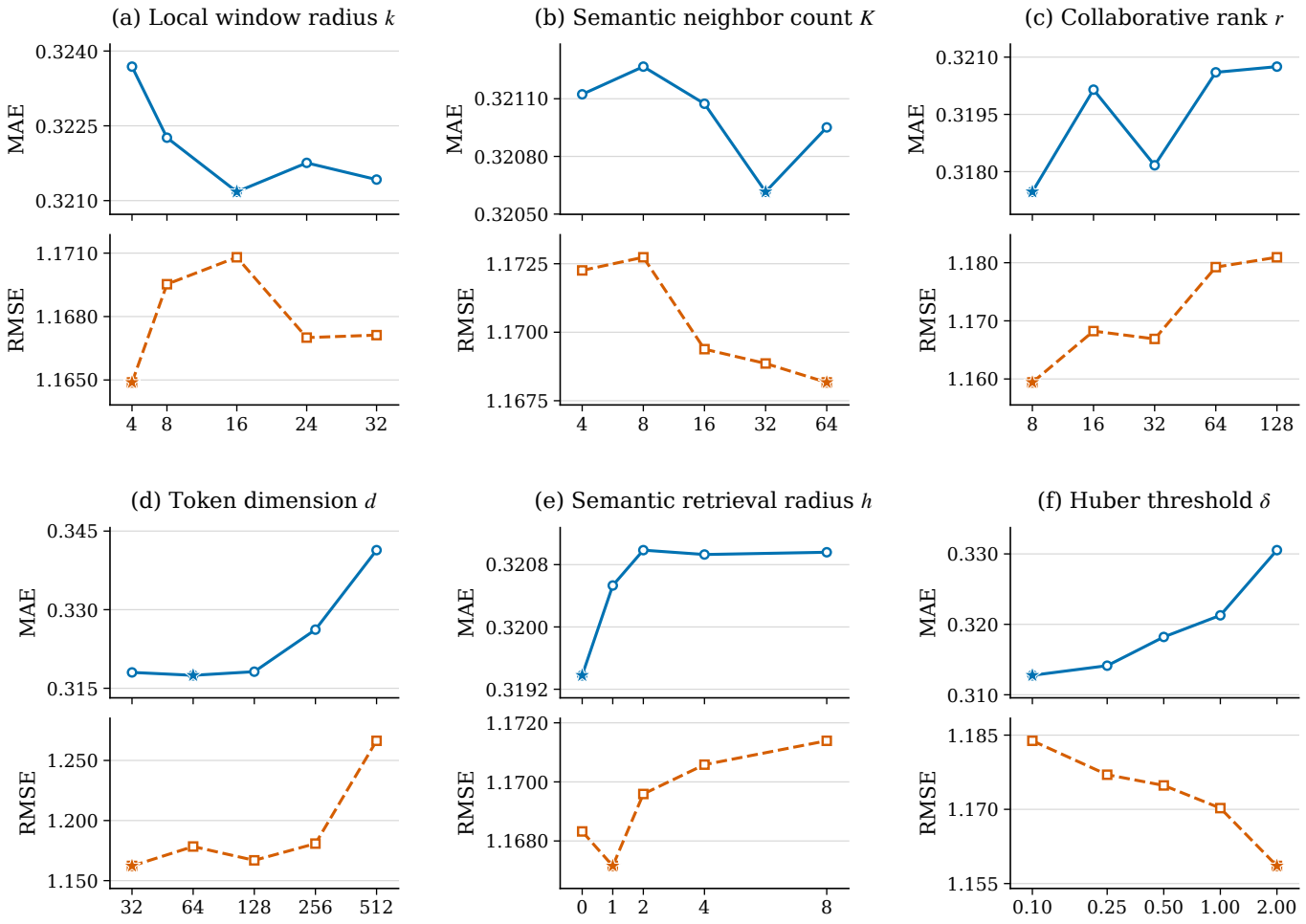



Fig. 2. Hyperparameter sensitivity at 10% observation density and a batch size of 8,192. Panels (a)–(f) vary the local window radius $k$, semantic neighbor count $K$, collaborative rank $r$, token dimension $d$, semantic retrieval radius $h$, and Huber threshold $\delta$, respectively. Each panel shows MAE above RMSE; lower values are better. Stars mark the lowest observed value for each metric. The horizontal axes for $K, r, d, \delta$ are logarithmic. Vertical scales vary across panels.

**Semantic neighbor count.** Performance varies modestly across $K \in \{4, 8, 16, 32, 64\}$. Both metrics remain within 0.4% of their respective minima over the tested range. The lowest MAE occurs at $K = 32$, whereas the lowest RMSE occurs at $K = 64$. Increasing the neighborhood beyond 32 therefore produces a small RMSE improvement accompanied by a slight MAE increase. The limited variation indicates low sensitivity to neighbor count under this reference configuration.

**Collaborative rank.** Increasing the latent dimension does not consistently improve prediction. The smallest tested rank, $r = 8$, achieves the lowest MAE and RMSE, at 0.3175 and 1.1594, respectively. At $r = 128$, these errors increase by 1.03% and 1.86%. Although the intermediate results are nonmonotonic, larger factor representations provide no advantage over the compact representation in this sweep. This observation supports using a restrained collaborative capacity under the evaluated sparsity and optimization settings.

**Token dimension.** MAE remains close across $d \in \{32, 64, 128\}$, reaching its minimum at $d = 64$, while RMSE is lowest at $d = 32$. Increasing $d$ to 256 and 512 degrades both metrics relative to $d = 64$. At $d = 512$, MAE and RMSE reach 0.3414 and 1.2664, respectively. These results support compact token representations under the reference configuration and show that increasing representational width

alone does not yield better predictions.

**Semantic retrieval radius.** We vary the retrieval radius $h$ from 0 to 8, where $h = 0$ restricts supporting observations to the target time slot. This setting achieves the lowest MAE, 0.3194. Allowing one adjacent time slot on either side produces a slightly lower RMSE, but extending the radius to two or more slots yields higher errors than $h = 0$ in both metrics. Same-slot retrieval consequently provides the lowest MAE and near-minimum RMSE in this sweep, consistent with its use in the final model.

**Huber threshold.** The threshold produces a clear trade-off between the two metrics. Increasing $\delta$ from 0.1 to 2.0 raises MAE from 0.3128 to 0.3305 while reducing RMSE from 1.1839 to 1.1586, corresponding to changes of +5.68% and $-2.14\%$, respectively. This behavior is consistent with the wider quadratic region and greater influence of large residuals associated with a larger threshold. Smaller thresholds favor absolute-error accuracy, whereas larger thresholds favor squared-error accuracy. The adopted value $\delta = 1.0$ provides an intermediate operating point along this observed trade-off.

## 6 Conclusion

We presented DMVF, a disentangled multi-view framework for sparse time-aware QoS prediction. The framework organizes persistent interaction patterns, temporal variation, and contextual evidence into three dedicated views. Collaborative factors, entity identities, and interaction statistics characterize long-term behavior, while absolute-time embeddings and compact local summaries capture temporal context. The semantic view fuses multi-frequency geographic encodings with statistics of same-slot training observations retrieved using frozen language-model similarities and adapter-refined aggregation weights. Hierarchical attention integrates these representations, and a residual predictor learns a correction to a stable median anchor.

Experiments on Supplement_WSDREAM_dataset across four observation densities demonstrate improvements over several evaluated baselines. DMVF achieves the lowest reported RMSE at 10% density and slightly higher training throughput than STF. STF achieves lower MAE at all four densities, while TUIPCC achieves lower RMSE at 15% and 20% densities. Ablation studies show the largest degradation when the time-independent view is removed and consistent benefits from temporal features. The semantic branch contributes a modest RMSE improvement accompanied by a small MAE trade-off. Sensitivity analysis further shows that increasing representation capacity does not consistently improve prediction, while the Huber threshold controls the balance between absolute and squared error.

Future work will investigate deeper integration of large language models into QoS prediction, including jointly updating language-model weights and prediction-network parameters during training. This direction aims to learn contextual representations and semantic relationships that are directly aligned with service-quality estimation. Parameter-efficient fine-tuning will also be explored to assess the predictive benefits of task-specific adaptation while managing training and memory costs.

## References


[1] Z. Zheng, H. Ma, M. R. Lyu, and I. King, "QoS-aware web service recommendation by collaborative filtering," *IEEE Transactions on Services Computing*, vol. 4, no. 2, pp. 140–152, 2011.

[2] Z. Zheng, Y. Zhang, and M. R. Lyu, "Investigating QoS of real-world web services," *IEEE Transactions on Services Computing*, vol. 7, no. 1, pp. 32–39, 2014.

[3] Q. Zhou, H. Wu, K. Yue, and C.-H. Hsu, "Spatio-temporal context-aware collaborative QoS prediction," *Future Generation Computer Systems*, vol. 100, pp. 46–57, 2019.

[4] J. Zhou, D. Ding, Z. Wu, and Y. Xiu, "Spatial context-aware time-series forecasting for QoS prediction," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 918–931, 2023.

[5] Y. Zhao, Y. Gui, L. Zhang, J. Zhang, M. Jin, and H. Wu, "STF: Steady and transient factorization for sparse time-aware QoS prediction," *IEEE Transactions on Services Computing*, 2026.

[6] H. Liu, Z. Zhang, H. Li, Q. Wu, and Y. Zhang, "Large language model aided QoS prediction for service recommendation," in *Proceedings of the 2025 IEEE International Conference on Software Services Engineering (SSE)*. IEEE, 2025.

[7] Z. Zheng, H. Ma, M. R. Lyu, and I. King, "Collaborative web service QoS prediction via neighborhood integrated matrix factorization," *IEEE Transactions on Services Computing*, vol. 6, no. 3, pp. 289–299, 2013.

[8] Y. Zhang, Z. Zheng, and M. R. Lyu, "WSPred: A time-aware personalized QoS prediction framework for web services," in *Proceedings of the 22nd IEEE International Symposium on Software Reliability Engineering*. IEEE, 2011, pp. 210–219.

[9] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 Embedding: Advancing text embedding and reranking through foundation models," arXiv preprint arXiv:2506.05176, 2025. [Online]. Available: https://arxiv.org/abs/2506.05176

[10] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng, "Fourier features let networks learn high frequency functions in low dimensional domains," in *Advances in Neural Information Processing Systems*, vol. 33, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/55053683268957697aa39fba6f231c68-Abstract.html

[11] T. Liang, M. Chen, Y. Yin, L. Zhou, and H. Ying, "Recurrent neural network based collaborative filtering for QoS prediction in IoV," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 3, pp. 2400–2410, 2022.

[12] G. Zou, T. Li, M. Jiang, S. Hu, C. Cao, B. Zhang, Y. Gan, and Y. Chen, "DeepTSQP: Temporal-aware service QoS prediction via deep neural network and feature integration," *Knowledge-Based Systems*, vol. 241, p. 108062, 2022.

[13] E. Tong, W. Niu, and J. Liu, "A missing QoS prediction approach via time-aware collaborative filtering," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3115–3128, 2022.

[14] R. Xiong, J. Wang, Z. Li, B. Li, and P. C. K. Hung, "Personalized LSTM based matrix factorization for online QoS prediction," in *Proceedings of the 2018 IEEE International Conference on Web Services (ICWS)*. IEEE, 2018, pp. 34–41.

[15] H. Liu, Y. Li, M. Tsang, and Y. Liu, "CoSTCo: A neural tensor completion model for sparse tensors," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. Association for Computing Machinery, 2019, pp. 324–334.

[16] H. Wu, S. Tian, B. Jin, Y. Zhao, and L. Zhang, "Effective graph modeling and contrastive learning for time-aware QoS prediction," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3513–3526, 2024.

[17] X. Wu, B. Shi, Y. Dong, C. Huang, and N. V. Chawla, "Neural tensor factorization for temporal interaction learning," in *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*. Association for Computing Machinery, 2019, pp. 537–545.